INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

Pruebas de carga con 1000 usuarios

y avances desde la entrega del 25%

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Objetivo del documento 3
2. Entorno y escenario de prueba 3
3. Prueba 1: 1000 usuarios en 1 minuto 3
   Interpretación 4
4. Prueba 2: cuánto tiempo aguanta los 1000 usuarios 4
   Cuánto aguantó 6
   Interpretación 6
5. Cuello de botella y cómo soportar 1000 y 5000 usuarios 7
6. Avances desde la entrega del 25% 7
7. Caídas e incidentes desde el 25% 10
8. Pendientes para completar el 80% 12

1. Objetivo del documento

Este documento presenta tres cosas: (1) la prueba de carga con k6 de 1000 usuarios que llegan en 1 minuto, en bloques de 100; (2) una prueba de aguante que mide cuánto tiempo resiste el sistema con esos 1000 usuarios conectados; y (3) lo que se agregó al proyecto desde la entrega del 25%.

Siguiendo la indicación del curso, se separa lo que se evalúa en la entrega del 80% de lo que se agregó como extra. Las pruebas se hicieron en la laptop del equipo porque el servidor en AWS aún no está creado; por eso sus resultados muestran el límite de una sola máquina y no el del sistema desplegado.

2. Entorno y escenario de prueba

Tabla 1. Entorno de las pruebas

Elemento	Configuración
Equipo	Laptop del equipo: 20 hilos de CPU y 15,6 GB de RAM (Windows 11). k6 corre en la misma laptop.
API	Docker: Nginx 1.27 + PHP-FPM (20 procesos) + OPcache + caché de Laravel (config, rutas y eventos)
Base de datos y caché	MySQL 8.4 y Redis 7 en Docker
Herramienta	k6 2.2 con el escenario tests/k6/reservas.js (recepción)
Rate limits	Elevados solo durante la prueba para que los 429 no cuenten como errores; se restauraron al terminar
Tiempo máximo por petición	10 s en Nginx y PHP (lo que dure más se corta con 503/504)

Escenario: cada usuario virtual es un recepcionista. En cada iteración: 70% consulta (habitaciones disponibles, reservas y tipos), 20% inicia sesión y 10% crea una reserva; luego espera entre 1 y 2 segundos. Metas del 80%: p95 menor a 800 ms y menos de 1% de errores.

3. Prueba 1: 1000 usuarios en 1 minuto

Los usuarios suben de 100 en 100 cada 5 segundos (1 s subiendo y 4 s sostenido por bloque) hasta llegar a 1000 en 1 minuto; luego bajan en 10 s.

```
bash tests/k6/correr-bloques.sh 1000 5
python docs/pruebas/bloques.py 1000:5
```

Tabla 2. Resultado total de la prueba 1

Peticiones	Pet./s	Mediana	p95	Errores	Meta
4,903	44.6	10.00 s	10.00 s	75.0 %	p95 < 0,8 s y < 1 %

Tabla 3. Prueba 1 por bloque de 100 usuarios (5 s cada uno)

Usuarios	Peticiones	Pet./s	p95	Errores
100	102	20.4	3.69 s	0.0 %
200	175	35.0	4.10 s	0.0 %
300	200	40.0	4.97 s	0.0 %

Usuarios	Peticiones	Pet./s	p95	Errores
400	218	43.6	6.63 s	0.0 %
500	225	45.0	8.64 s	0.0 %
600	214	42.8	9.13 s	0.0 %
700	268	53.6	10.01 s	85.1 %
800	322	64.4	10.01 s	100.0 %
900	355	71.0	10.00 s	100.0 %
1000	428	85.6	10.00 s	100.0 %

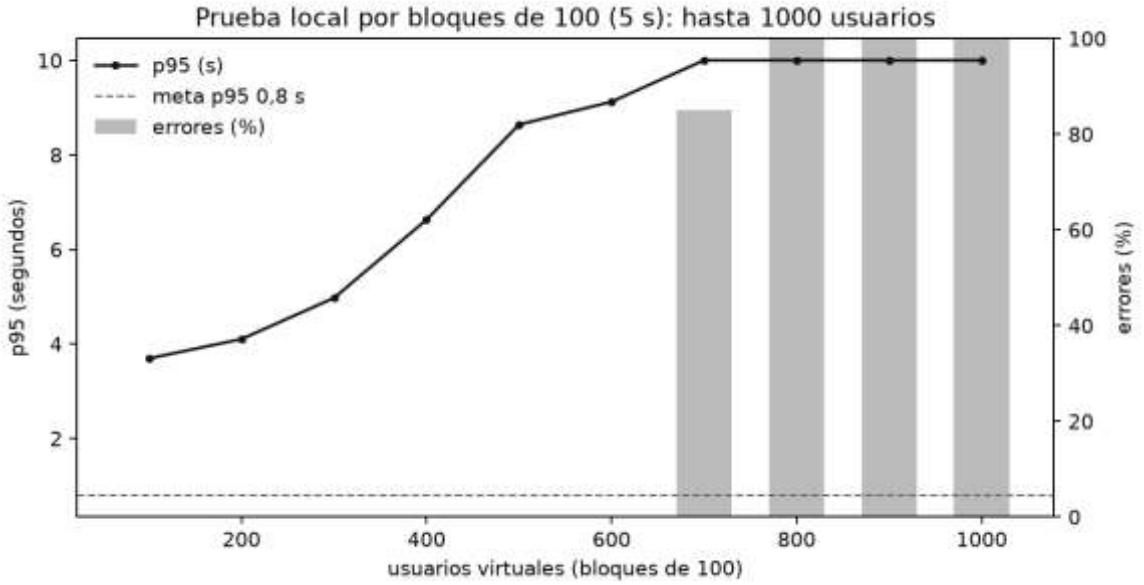


Figura 1. Prueba 1: p95 y errores en cada bloque de 100 usuarios

Interpretación

Hasta 600 usuarios no hubo errores, pero la meta de tiempo no se cumple desde el primer bloque: el p95 sube de 3.7 s a 9.1 s porque la laptop no atiende más de unas 45 peticiones por segundo y el resto espera en cola. Con 700 usuarios las peticiones esperan más de 10 s y Nginx las corta: ese es el punto de quiebre local. Desde ahí las «peticiones por segundo» suben porque los cortes responden rápido, no porque se atienda a más gente.

4. Prueba 2: cuánto tiempo aguanta los 1000 usuarios

Los 1000 usuarios llegan en 1 minuto (bloques de 100) y se mantienen conectados 3 minutos más. Se mide cuánto tiempo responde sin errores, cómo se comporta con los 1000 sostenidos y si la API se recupera sola al terminar.

```
bash tests/k6/correr-aguante.sh 1000
python docs/pruebas/aguante.py 1000
```

Tabla 4. Resultado total de la prueba 2

Peticiones	Pet./s	Mediana	p95	Errores	Duración
30,916	102.8	8.35 s	12.04 s	95.3 %	1 min de subida + 3 min sostenidos

Tabla 5. Prueba 2 en tramos de 10 segundos

Desde (s)	Usuarios	Peticiones	Pet./s	p95	Errores
0	200	276	27.6	4.08 s	0.0 %
10	400	447	44.7	6.16 s	0.0 %
20	500	453	45.3	8.35 s	0.0 %
30	700	469	46.9	10.01 s	49.7 %
40	900	687	68.7	10.01 s	100.0 %
50	1000	863	86.3	10.00 s	100.0 %
60	1000	944	94.4	10.00 s	100.0 %
70	1000	970	97.0	10.01 s	100.0 %
80	1000	897	89.7	10.00 s	100.0 %
90	1000	967	96.7	10.01 s	100.0 %
100	1000	943	94.3	10.00 s	100.0 %
110	1000	1320	132.0	10.01 s	100.0 %
120	1000	1608	160.8	12.04 s	100.0 %
130	1000	1601	160.1	12.04 s	100.0 %
140	1000	1567	156.7	12.04 s	100.0 %
150	1000	1512	151.2	12.04 s	100.0 %
160	1000	1545	154.5	12.04 s	100.0 %
170	1000	1596	159.6	12.04 s	100.0 %
180	1000	1646	164.6	12.04 s	100.0 %
190	1000	1564	156.4	12.04 s	100.0 %
200	1000	1698	169.8	12.04 s	100.0 %
210	1000	1613	161.3	12.04 s	100.0 %
220	1000	1553	155.3	12.04 s	100.0 %
230	1000	1600	160.0	12.05 s	100.0 %
240	999	1459	145.9	12.04 s	100.0 %
250	683	642	64.2	12.05 s	100.0 %
260	350	324	32.4	12.06 s	100.0 %
270	134	99	9.9	11.03 s	100.0 %
280	0	1	0.1	10.00 s	100.0 %

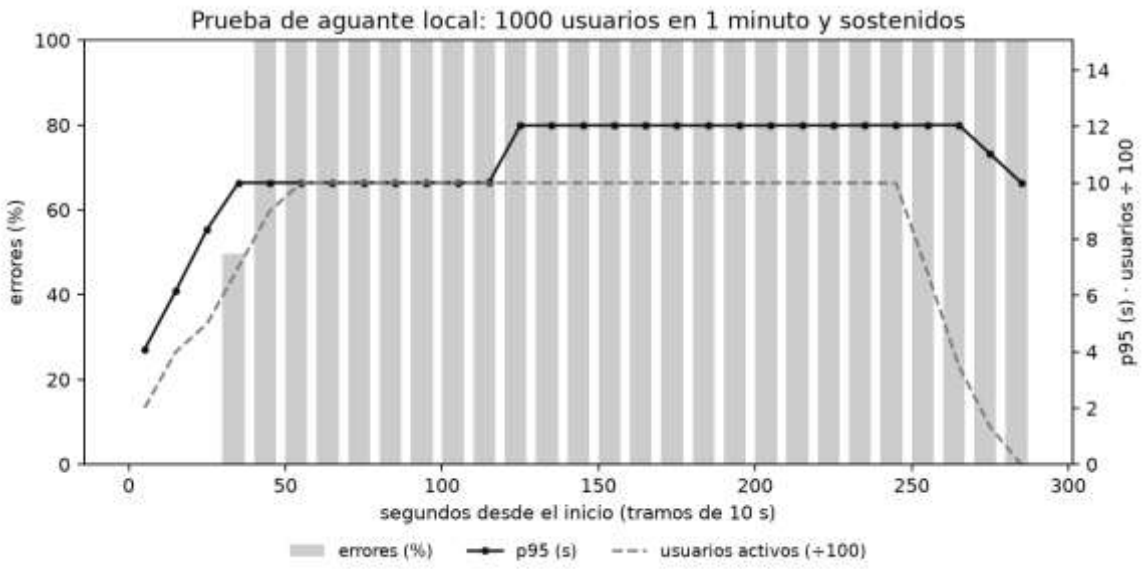


Figura 2. Prueba 2: usuarios activos, p95 y errores cada 10 segundos

Cuánto aguantó

- **Sin ningún error:** los primeros 35 segundos.
- **Errores por encima del 1%:** desde el segundo 30, con unos 700 usuarios conectados.
- **Con los 1000 usuarios sostenidos:** 19 de 19 tramos de 10 s superaron el 1% de errores.
- **Recuperación:** al terminar la prueba, /health respondió 503 en 5 intentos (cada uno esperó 10 s) y volvió a 200 unos 75 s después; desde ahí respondió en menos de 0,2 s.

Interpretación

El sistema aguanta unos 35 segundos sin errores mientras los usuarios van llegando; el problema aparece al pasar de unos 500 a 700 usuarios. Con los 1000 sostenidos no se recupera: todos los tramos tienen 100% de errores aunque la laptop sigue atendiendo unas 45 peticiones por segundo.

La razón es la cola: las peticiones que no se pueden atender en el momento esperan en orden de llegada. Cuando la cola ya supera los 10 s, cada petición llega al frente después de su tiempo límite y Nginx la corta, así que el trabajo se hace para respuestas que nadie recibe. La API no se cae (no se reinicia ni pierde datos): al terminar la prueba vacía la cola y unos 75 s después vuelve a responder en menos de 0,2 s.

Corrección propuesta además de más capacidad: limitar la cola (load shedding). Si Nginx y PHP-FPM aceptan solo las peticiones que pueden atender a tiempo y responden al resto de inmediato con 503 y Retry-After (pantalla que la aplicación ya maneja), los usuarios atendidos reciben respuesta en pocos segundos en lugar de que todos esperen y fallen.

5. Cuello de botella y cómo soportar 1000 y 5000 usuarios

Con el ritmo del escenario (una acción cada 1 a 2 segundos por usuario), 1000 usuarios necesitan unas 1500 peticiones por segundo y 5000 unas 7500; la laptop atiende unas 45. Las causas encontradas y las mejoras propuestas son:

Tabla 6. Mejoras propuestas por nivel

Nivel	Qué se hace	Efecto esperado
1. Ajustes locales	Caché y rate limiter en Redis; caché corta de tipos y disponibilidad; menos escrituras de Sanctum por petición; bcrypt costo 10; k6 desde otro equipo	2 a 4 veces más (por medir)
1b. Limitar la cola	Nginx (limit_conn) y PHP-FPM (listen.backlog) aceptan solo lo que pueden atender a tiempo; el exceso recibe 503 con Retry-After al instante	Los atendidos responden a tiempo en vez de fallar todos
2. Laravel en memoria	Laravel Octane (FrankenPHP o Swoole): el framework no se recarga en cada petición	3 a 5 veces por CPU
3. Servidor en AWS	Balanceador con Auto Scaling, RDS con réplica de lectura, ElastiCache y k6 desde EC2 generadoras (Terraform ya preparado)	Escala con el número de instancias

La prueba del 80% en el servidor se hará con el mismo script (PROFILE=bloques VUS=1000 y VUS=5000) cuando se cree la infraestructura en AWS, y se compararán los resultados con los de este documento.

6. Avances desde la entrega del 25%

Lista de lo que se agregó o mejoró después del informe del 25% (27/10/2026). La última columna indica si responde a un punto evaluado del 80% o si es un extra, como pide el curso.

Tabla 7. Avances desde la entrega del 25%

Avance	Qué se hizo	Evidencia	Tipo
Respaldos (backups)	Respaldo de MySQL comprimido y cifrado (AES-256) en un repositorio privado de GitHub; prueba de restauración con las 5 tablas iguales	infra/scripts/backup-mysql-to-github.sh y restore-from-github.sh	Evaluado 80%
Pruebas con 1000 usuarios	Pruebas por bloques de 100, de 1 minuto y de aguante, con tabla y gráfico por bloque	tests/k6/correr-bloques.sh, correr-aguante.sh; docs/pruebas/	Evaluado 80%
Rate limit del login	Tras una contraseña incorrecta hay que esperar 30 s, y la espera se duplica en cada fallo (hasta 15 min); cuenta regresiva en el login	LoginLockout; AuthTest; tests/k6/rate-limit.js	Evaluado 80%
Ingreso con correo (Google)	Revisión de Google OAuth y corrección del regreso al primer dominio del frontend	GoogleAuthController; ClientPortalTest	Evaluado 80%
Múltiples tipos de usuario	Un solo Programador (nadie crea otro ni le quita el rol); recorrido probado por rol (47 pasos)	docs/recorridos; pruebas de roles	Evaluado 80%
Perfil del cliente	DNI y celular opcionales al registrarse; «Mi perfil» para editarlos; al reservar se piden si faltan o se avisa a nombre de quién queda	ProfilePage, BookPage; ClientPortalTest, client.test.tsx	Mejora

Avance	Qué se hizo	Evidencia	Tipo
Mejora de rendimiento medida	Nginx + PHP-FPM + OPcache + caché de Laravel: 4 veces más peticiones y p95 18 veces menor que con php -S	docs/pruebas/README.md	Extra (uso de recursos)
Pruebas de clientes del portal	Escenario k6 de clientes (buscan, ven y crean reservas) y comando k6:limpiar	tests/k6/portal.js	Extra
Modo oscuro e inglés	Configuración con tema claro, oscuro o del sistema e idioma español o inglés; la API responde en el idioma elegido	SettingsPage, i18n; i18n.test.tsx	Extra
Tutorial de bienvenida	Al primer ingreso se ofrece un recorrido que oscurece la pantalla y resalta cada sección del rol; se repite desde Configuración	Tour.tsx; POST /auth/tutorial; tour.test.tsx	Extra
Filtros del tablero	El tablero de habitaciones del panel filtra por piso y estado y muestra las 70 habitaciones (antes 50)	RoomRack.tsx; rack.test.tsx	Extra
Documentación	Diagrama de la base generado desde el esquema real; tutoriales de k6 y MySQL Workbench; informe de pruebas k6 con capturas	docs/diagramas, docs/pruebas	Extra
Registro de caídas	Cada caída, incluso local, con motivo y solución (sección 7)	docs/incidentes.md	Evaluado

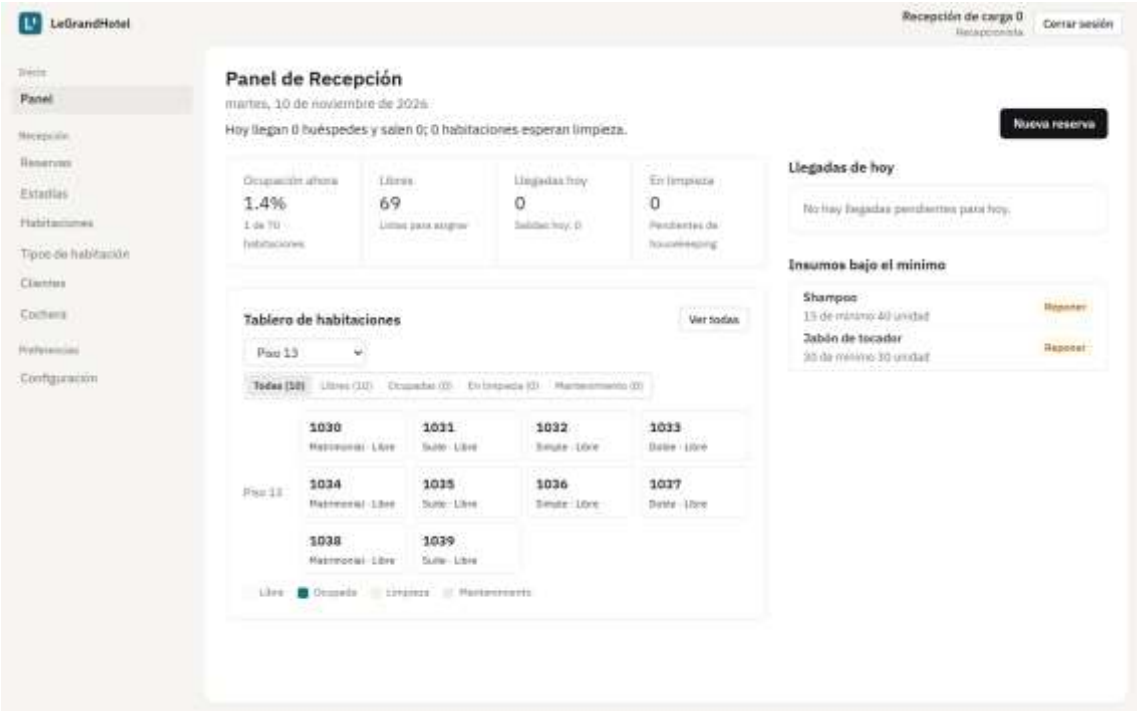


Figura 3. Tablero de habitaciones con los filtros por piso y estado

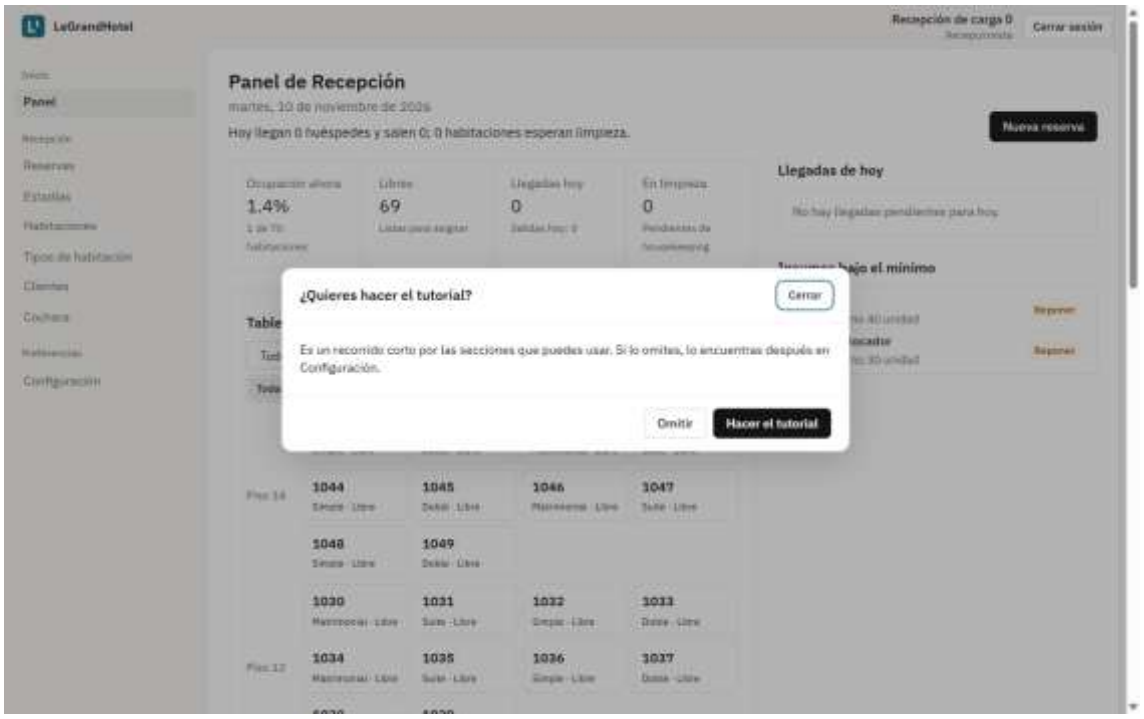


Figura 4. Primer ingreso: se ofrece hacer el tutorial u omitirlo

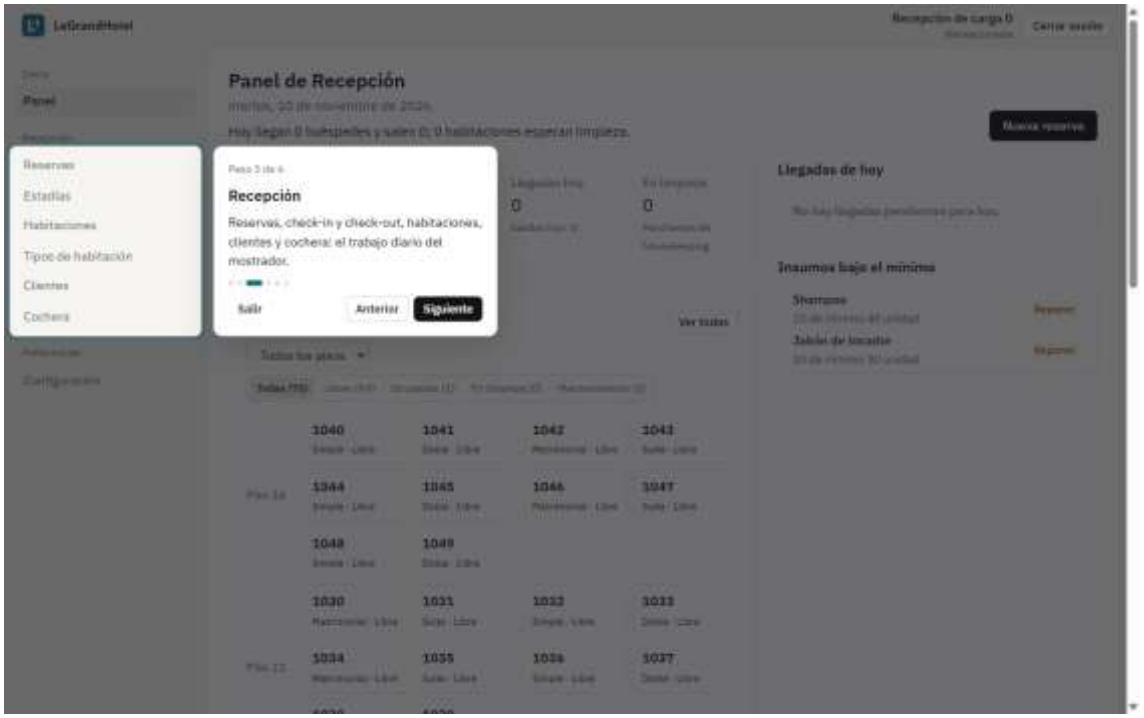


Figura 5. Tutorial de bienvenida: la pantalla se oscurece y se resalta cada sección

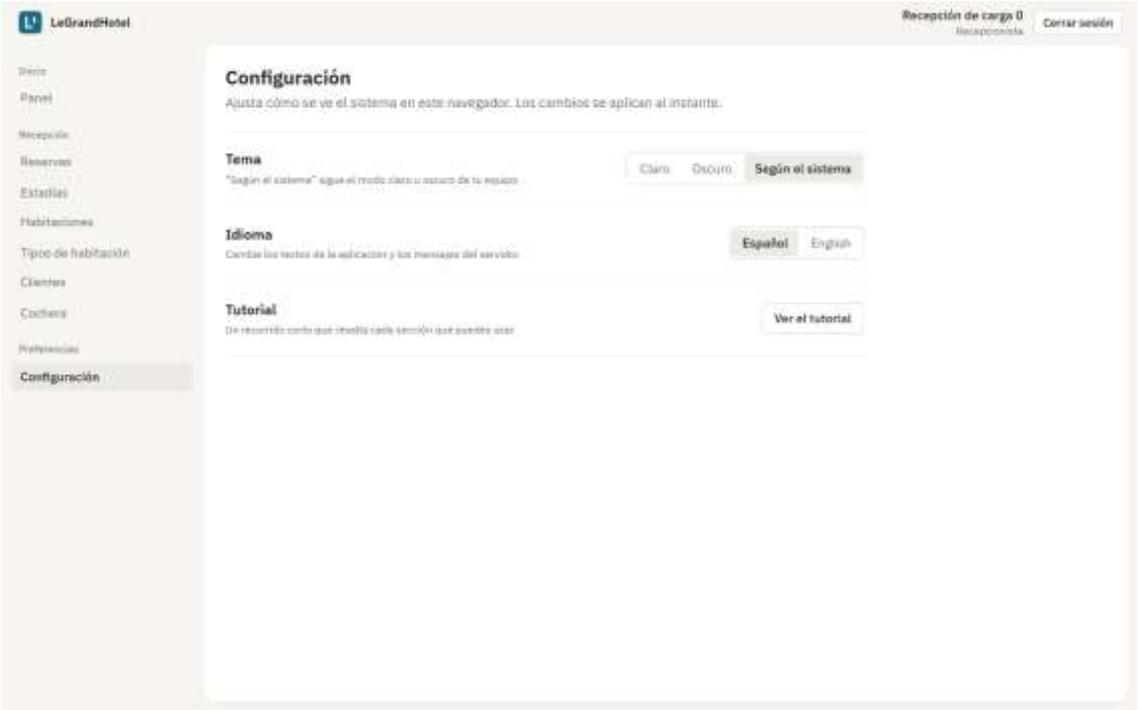


Figura 6. Configuración: tema, idioma y tutorial

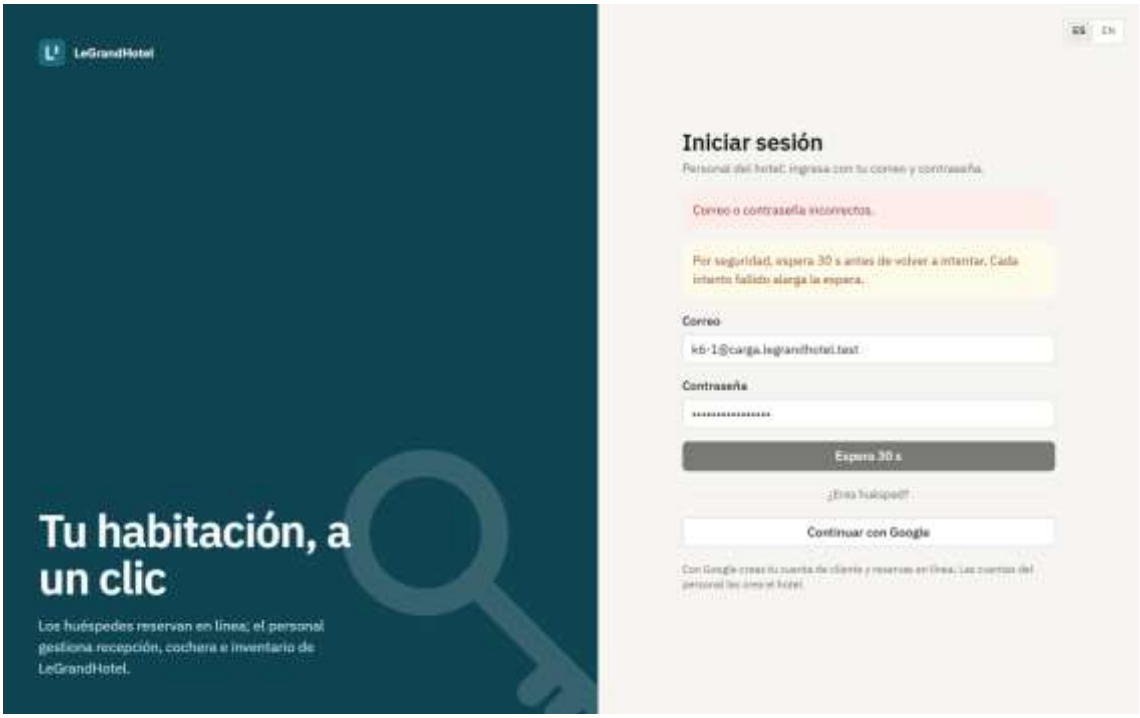


Figura 7. Login: cuenta regresiva después de una contraseña incorrecta

7. Caídas e incidentes desde el 25%

Todas las caídas se registran en docs/incidentes.md con su motivo y solución, aunque sean en local o durante una prueba.

Tabla 8. Incidentes registrados desde el 28/10/2026

Fecha	Entorno	Qué falló	Motivo	Solución	Tiempo
2026-10-28	Local (desarrollo, MySQL)	php artisan migrate falló con «No se puede establecer una conexión… denegó expresamente dicha conexión» (SQLSTATE 2002)	Docker Desktop no estaba encendido, así que el contenedor legrandhotel-mysql-1 tampoco	Se abrió Docker Desktop y docker start legrandhotel-mysql-1; la migración corrió bien. Las pruebas Pest no se afectaron (usan SQLite en memoria)	~10 min
2026-10-28	Local (pruebas k6)	rate-limit.js falló 2 de 2 checks con «connection refused»	Igual que el 27/10: php -S escucha solo en IPv6 (::1) y k6 resolvió localhost a 127.0.0.1	BASE_URL=http://[::1]:8000/api/v1; la corrida pasó 5 de 5 checks	~1 min
2026-11-10	Local (desarrollo, MySQL)	php artisan migrate (columnas del tutorial) falló con SQLSTATE 2002; docker ps no conectaba con el motor	Docker Desktop estaba apagado otra vez (igual que el 28/10); un primer Start-Process no lo dejó corriendo	Se volvió a abrir Docker Desktop, docker start legrandhotel-mysql-1 y la migración corrió bien. Pest no se afectó (SQLite en memoria)	~8 min
2026-11-10	Local (stack Docker: Nginx + PHP-FPM)	/api/v1/health devolvió 503 SERVICE_UNAVAILABLE dos veces seguidas	Recién encendido Docker, PHP-FPM tardó en responder la primera petición (arranque en frío, sin OPcache caliente) y Nginx cortó por tiempo: «upstream timed out … fastcgi://…:9000» en docker logs legrandhotel-nginx-1	Sin cambios: a los ~2 min respondió 200 con base y caché «ok». Antes de una prueba k6, hacer una petición de calentamiento	~2 min
2026-11-10	Local (prueba k6, 1000 usuarios en bloques de 100)	La API respondió 503/504 al 78% de las peticiones; p95 de 11 s	La laptop llega a ~40–60 pet./s (20 procesos PHP-FPM, login con bcrypt); con más usuarios las peticiones esperan más de 10 s y Nginx corta («upstream timed out»). La base siguió sana	Ninguno en local (es el límite de la laptop). En AWS: más CPU, varias instancias con Auto Scaling, RDS y Redis. Detalle en docs/pruebas/README.md	~4 min (durante la prueba)
2026-11-10	Local (prueba k6)	La primera corrida de 1000 usuarios se detuvo en setup() con 401 al iniciar sesión	Las 50 cuentas de carga se habían creado antes con otra clave y LoadTestUsersSeeder no la actualizaba (insertOrIgnore)	El seeder ahora hace upsert y vuelve a poner la clave de la prueba	~2 min
2026-11-10	Local (prueba k6, 1000 usuarios en 1 minuto)	Desde el bloque de 700 usuarios la API respondió 503/504 (85–100% de errores por bloque); 75% en toda la corrida	Mismo límite de la laptop (~45 pet./s): con 700 usuarios las peticiones esperan más de 10 s y Nginx corta. Hasta 600 usuarios no hubo errores	Ninguno en local; se repetirá en AWS con Auto Scaling. Tabla por bloque en docs/pruebas/README.md	~30 s (durante la prueba)
2026-11-10	Local (stack Docker)	Después de las pruebas k6 la web quedaba en «Demasiadas peticiones» y la API respondía 429 a todo, aun sin carga	api/.env tenía RATE_LIMIT_API=, RATE_LIMIT_LOGIN= y RATE_LIMIT_REGISTER= vacías: (int) env('RATE_LIMIT_API', 60) no usa el valor por defecto con una línea vacía y el límite quedaba en 0 (cabecera X-RateLimit-Limit: 0)	config/hotel.php usa el valor por defecto si la variable está vacía (prueba RateLimitConfigTest); api/.env vuelve a 5 / 3 / 60 como .env.example	~10 min

8. Pendientes para completar el 80%

- Crear el entorno en AWS (Terraform listo) y desplegar el proyecto con el pipeline deploy.yml.
- Repetir en el servidor las pruebas de 1000 y 5000 usuarios con k6 desde EC2 generadoras.
- Configurar en AWS los respaldos automáticos (snapshots de RDS y backup-mysql-to-s3.sh).
- Guardar las credenciales de Google en el servidor y registrar la URL de regreso del dominio real.